

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
df = pd.read_csv("Mall_Customers.csv")
df.head()
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.shape
```

```
(200, 5)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   CustomerID            200 non-null   int64
1   Gender                200 non-null   object
2   Age                   200 non-null   int64
3   Annual Income (k$)    200 non-null   int64
4   Spending Score (1-100) 200 non-null   int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.isnull().sum()
```

```

CustomerID    0
Gender        0
Age           0
Annual Income (k$)  0
Spending Score (1-100)  0
```

```
dtype: int64
```

```
duplicates = df.duplicated().sum()
print(duplicates)
```

```
0
```

```
df.describe()
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
#features for clustering
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print(X.head())
```

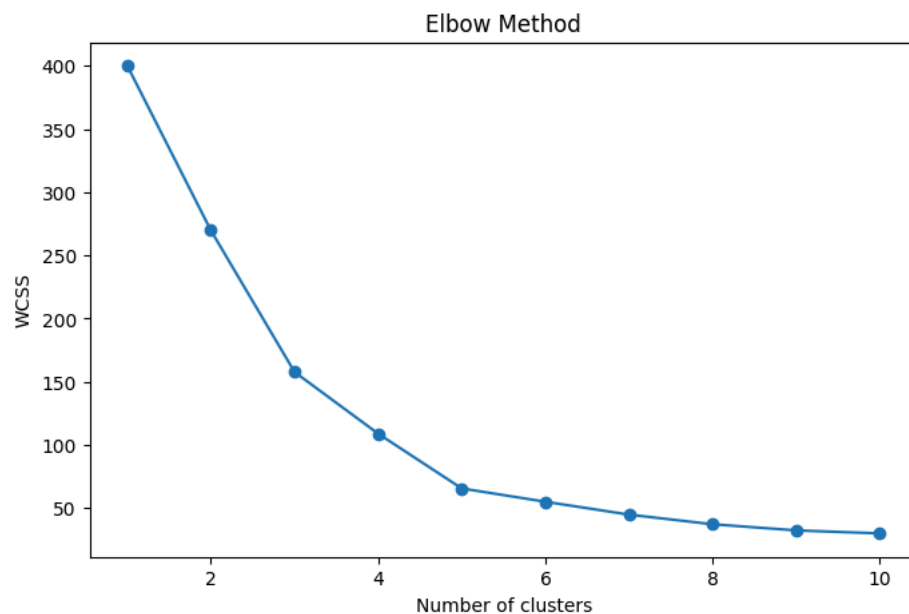
```
Annual Income (k$)  Spending Score (1-100)
0                  15                  39
1                  15                  81
2                  16                   6
3                  16                  77
4                  17                  40
```

```
#Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(X_scaled[:5])
```

```
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]
```

```
#Elbow method
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters = i,
                    random_state = 42,
                    n_init = 10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
```

```
#plot elbow method
plt.figure(figsize =(8,5))
plt.plot(range(1,11),wcss,marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters")
plt.ylabel('WCSS')
plt.show()
```



```
#train Kmeans
kmeans = KMeans(
    n_clusters=5,
    random_state = 42,
    n_init = 10
)
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters

#display cluster labels
print(df[['CustomerID', 'Cluster']].head(10))
```

	CustomerID	Cluster
0	1	4
1	2	2
2	3	4
3	4	2
4	5	4
5	6	2
6	7	4
7	8	2
8	9	4
9	10	2

```
#no.of customers in each cluster
print(df['Cluster'].value_counts())

#visulaize clusters
plt.figure(figsize = (8,6))

plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
    c= df['Cluster']
)

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Customer segmentation using k-means')
plt.show()
```

```
Cluster
0    81
1    39
3    35
4    23
2    22
Name: count, dtype: int64
```



****What is Clustering? **** It is a unsupervised machine learning technique that groups similar data points into clusters based on their characteristics

Difference Between Supervised and Unsupervised Learning? supervised learning excels in predictive tasks with known outcomes, while unsupervised learning is ideal for discovering relationships and trends in raw data.

What is K-Means? K-Means is an unsupervised clustering algorithm that divides data into K clusters based on similarity.

What is a Centroid? A centroid is the center point of a cluster.

Explain Elbow Method. The elbow method is a visual heuristic used in machine learning to determine the optimal number of clusters(k) for algorithms like K-Means